Next.js Internal Coding Standard

---

1. Introduction

- **Purpose**

  This document defines **mandatory coding standards and architectural rules** for all React-based projects within the organization. Its goal is to ensure **consistency, scalability, maintainability, and performance** across teams and projects.

- **Scope**

  These standards apply to:

  - Web applications built using **Next.js**
  - Projects using TypeScript or JavaScript
  - SaaS applications, Internal tools, dashboards, and customer-facing apps

- **Supported projects**

  - Next.js (latest stable)
  - App Router architecture (**Recommended**)
  - Next.js + Webpack (custom builds)
  - Next.js + Turbopack (**Recommended**)
  - TypeScript (mandatory)

- **Audience**

  - Frontend Developers
  - Full-stack Developers
  - Tech Leads & Architects
  - Code Reviewers

---

2. Prerequisites & Setup

- **Node.js version requirement**
    - Minimum Required: Node.js ≥ 20.19 or higher, or 22.12 or higher.
    - Preferred Version: Node.js 22.x or current LTS
    - Environment Consistency:
    -
    - The Node.js version must be identical across:
        - Local development
        - CI pipelines
        - Production environments
- **Must be aligned across local, CI, and production environments**

  Step 1: The `.nvmrc` file must exist at the project root and define the Node.js version to be used across all environments. The file **must specify either a major version** (e.g., `22`) **or an exact version** (e.g., `22.11.0`), depending on the required level of version strictness.

  Step 2: `package.json` Engine Enforcement

    - The `package.json` file **must** declare the supported Node.js versions using the `engines` field.

```JSON
{
  "engines": {
    "node": ">=18 <23"
  }
}
```

  Step 3: `Local Development`

    - Developers **must** use `nvm` (or an equivalent Node version manager).

```bash
nvm install
nvm use
```

Step 4: CI Configuration

- CI pipelines **must** read the Node.js version directly from `.nvmrc`
  Example (GitHub Actions):

```yaml
- uses: actions/setup-node@v4
  with:
    node-version-file: '.nvmrc'
```

Step 5: Production Environment

- Production servers **must** use `nvm` (or Volta) and load the Node.js version from `.nvmrc`

```bash
nvm install
nvm use
```

- **Package manager**
  - Preferred: `npm or bun`
  - Allowed: `pnpm` or `yarn` (project-dependent)
  - Lock files must be committed

- **Environment setup**
    - `.env.example` is mandatory
    - No secrets allowed in `.env` files committed to git
    - Environment variables must be accessed via a centralized config

- **Build tool (Turbopack / Webpack)**
    - Default: Turbopack
    - Custom Webpack requires architecture approval

---

3. Boilerplate Reference

- **Internal starter repository**
    - All new projects must start from the internal React starter
    - Custom setups require approval
    - Starter Repo Link

- **Branch rules**

```
Branch → Environment Mapping

main        → production
develop     → staging
feature/*   → development
fix/*       → development
hotfix/*    → production
```

- **Versioning**
    - Semantic Versioning (`MAJOR.MINOR.PATCH`)
    - Tags are mandatory for production releases

---

4. Project Structure

Example:

`/src`

`/app`

- **Purpose:** Contains Next.js routes. Each folder represents a route.
- **Example:**

```
├─ app
│   ├─ (public)
│   │   ├─ page.tsx
│   │   └─ layout.tsx
│   │
│   ├─ (auth)
│   │   ├─ login
│   │   └─ register
│   │
│   ├─ dashboard
│   │   ├─ layout.tsx
│   │   └─ page.tsx
```

`/assets`

- **Purpose:** Static assets such as images, icons, and fonts.
- **Example:**

```
/assets
├─ images
│  ├─ logo.svg
│  └─ banner.png
│
└─ icons
   ├─ index.ts
   └─ icon.tsx
```

`/components/ui` (Pure UI components for shadcn)

- **Purpose:** Reusable, framework-level UI primitives (e.g., shadcn-style components).
- **Example:**

```
/components/ui
  |  input.tsx
  └─ button.tsx
```

`/components`

- **Purpose:** Feature-level pure UI components (no logic, no API calls).

- **Example:**

```
/components/user-management
  | user-action.tsx
  └ user-card.tsx
```

/containers

- **Purpose:** Smart components handle data, provide hooks, and coordinate screens.
- **Example:**

```
containers/user-management/
├ users/
|  └ index.tsx
├ add-edit-user/
|  └ index.tsx
```

/integrations (3rd party integrations like Stripe, GA4, google-map-service, etc)

- **Purpose:** Third-party SDK and platform integrations.
- **Example:**

```
/integrations/ga4
  └ index.ts
```

/shared (Project-specific custom components)

- **Purpose:** Project-specific reusable components that are not generic UI primitives.
- **Example:**

```
/shared/empty-state
  └ empty-state.tsx
```

/hooks (Global hooks)

- **Purpose:** Global, reusable hooks not tied to a specific feature.
- **Example:**

```
/hooks
 ├ use-local-storage.ts
 └ use-debounce.ts
```

/services (API configuration, Interceptors, and Business services)

- **Purpose:** API configuration, interceptors, and business services.
- **Example:**

```
/services
  ├ index.ts
  └ api-client.ts
```

/store

- **Purpose:** Global state management (Redux / Zustand).
- **Example:**

```
/store
  ├ slice
  |   ├ auth.slice.ts
  |   ├ users.slice.ts
  |   ├ products.slice.ts
  |   ├ index.ts
  └ index.ts
```

/styles (Global styles)

- **Purpose:** Global styles and theme configuration.
- **Example:**

```
/styles
  └ globals.css
```

/types

- **Purpose:** Shared global TypeScript types and interfaces.
- **Example:**

```
/types
  ├ auth.types.ts
  └ user-access.types.ts
```

/utils

- **Purpose:** Helper functions, constants, and utility logic.
- **Example:**

```
/utils
├ functions
├ validations
└ constants
```

Rules:

- No API calls inside /components
- No business logic inside /pages
- Each folder has a single responsibility

Note: `/styles` - This directory must not be created when using shadcn/ui with Tailwind CSS.
It may be added only for project-specific global styles, theme overrides, or exceptional styling requirements.

---

5. Coding Conventions

- **Naming Rules**
    - Components: `PascalCase`
    - Hooks: `useCamelCase`
    - Variables & functions: `camelCase`
    - Constants: `UPPER_SNAKE_CASE`
    - Files match exported entity names
    - Use `kebab-case` for file names
- **Formatting (ESLint + Prettier)**
    - ESLint + Prettier are mandatory
    - Preferred: `husky or pre-commit hooks`
    - No manual formatting overrides
    - Lint must pass in CI
- **TypeScript Rules**
    - `strict: true` is mandatory
    - `any` is not allowed
    - Use `unknown` instead of `any`
    - Explicit return types for public functions
    - interfaces for objects, types for unions

---

6. Layout & Routing Management

Next.js uses **file-based routing and layout composition** through the `app` directory.
Layouts define the structural UI of the application and allow multiple routes to share common UI components.

Layouts must focus only on **structure and UI composition**, not business logic.

6.1 Layout Structure

Layouts define shared UI elements across multiple pages.

Typical layout responsibilities include:

- Navigation
- Sidebar
- Header
- Footer
- Page wrapper containers

Layouts allow consistent structure across route segments and enable hierarchical UI composition.

Structural Example

Rules

- Layouts must focus only on UI structure and composition.
- Layouts must not contain business logic.
- Layouts must not perform API calls.
- Layouts must render route content via the children prop.
- Layout nesting should remain shallow (recommended maximum: 2–3 levels).
- Layout components should remain reusable and predictable.

6.2 Route Groups

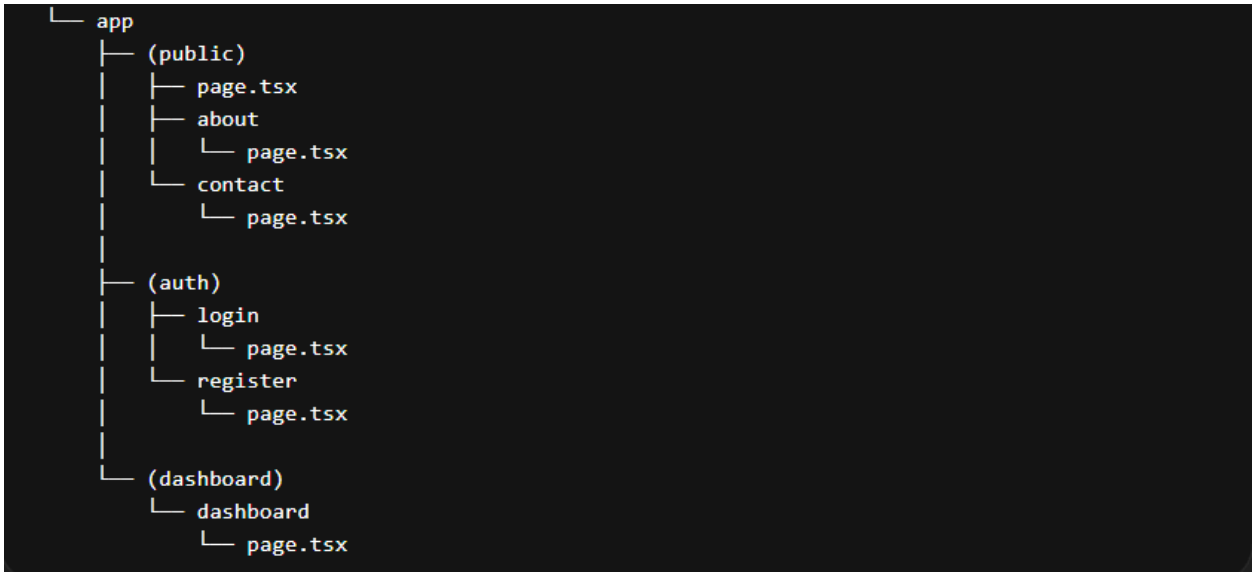
Route groups allow developers to organize routes logically without affecting the final URL structure.

They are used to group related routes within the application for better maintainability and architectural clarity.

Common grouping strategies include:

- Public routes
- Authentication routes
- Dashboard routes
- Admin modules

Structural Example



Rules

- Route groups must be used for logical organization only.
- Route groups must not affect the public URL structure.
- Groups should represent application domains or access levels.
- Avoid excessive nesting of route groups.
- Naming should clearly reflect the module or domain it represents.

6.3 Nested Routes

Next.js supports nested routing through folder hierarchy inside the `app` directory.

Nested routes enable feature modules to share layouts and maintain a consistent navigation structure, allowing for seamless integration and a unified user experience.

Nested routing is commonly used for:

- Dashboard modules
- Settings modules
- Resource management (users, orders, products)

- Feature subpages

Structural Example

Rules

- Nested routes must inherit the parent layout automatically.
- Deep nesting should be avoided (recommended maximum depth: 3 levels).
- Feature modules should remain self-contained.
- Each nested module must handle loading and error states appropriately.
- Nested routes should follow clear domain boundaries.

6.4 Middleware

Middleware allows execution of logic **before a request reaches the final route handler**.

It is commonly used for:

- Authentication checks
- Route protection
- Redirect handling

- Security enforcement
- Localization routing

Middleware operates at the edge of the application request lifecycle.

Structural Example

Rules

- Middleware must remain lightweight and fast.
- Middleware must not perform heavy operations.
- Middleware must not contain complex business logic.
- Database operations inside middleware should be avoided.
- Middleware should only handle routing, access control, and request validation.
- Complex logic should be delegated to services or backend APIs.

App Router File Responsibilities

- **layout.tsx** — Defines shared UI structure for a route segment, such as headers, navigation, sidebars, and page wrappers.

- **page.tsx** — Serves as the primary UI entry point for a route and renders the content of that page.
- **loading.tsx** — Displays a loading state while the route segment or its data is being fetched or rendered.
- **error.tsx** — Handles runtime errors within a route segment and shows a fallback error UI.
- **not-found.tsx** — Displays a custom “Not Found” interface when a requested resource or route does not exist.
- **template.tsx** — Provides a layout-like wrapper that re-renders when navigating between route segments.
- **route.ts** — Defines server-side API handlers for HTTP requests within the App Router.
- **middleware.ts** — Executes logic before a request reaches a route, typically used for authentication, redirects, and request validation.

---

7. Route Management & Best Practices

Next.js uses **file-based routing**, where routes are automatically generated from the folder structure inside the `app` directory.

Proper route organization ensures:

- predictable navigation
- clear access control
- maintainable architecture
- scalable module separation

Routes must follow defined access patterns such as **public, private, authentication, dynamic, and query-supported routes**.

7.1 Public Routes

Public routes are accessible to all users without authentication.

These routes are typically used for:

- landing pages
- marketing pages
- documentation
- legal pages
- publicly accessible resources

Structural Example

Rules

- Public routes must not require authentication.
- Sensitive user data must never be exposed.
- Public pages should prioritize performance and SEO.
- Public routes should be grouped logically when possible.

7.2 Private Routes

Private routes require authenticated access and are available only to logged-in users.

These routes represent core application functionality.

Typical examples include:

- dashboards
- user profile
- account settings
- order management
- billing systems

Structural Example

Rules

- Private routes must require authentication.
- Access control must be centralized (middleware or server validation).
- Unauthorized users must be redirected to the login.
- Protected resources must never be exposed without validation.

7.3 Auth Routes

Auth routes manage the authentication flow of the application.

These routes are responsible for:

- login
- user registration
- password recovery
- session initialization

Structural Example

Rules

- Auth routes must only be accessible to unauthenticated users.
- Logged-in users accessing auth routes should be redirected to the dashboard.
- Authentication logic must be handled via services or server logic.
- Auth pages must not expose sensitive authentication data.

7.4 Dynamic Routes

Dynamic routes allow parameterized URLs and are used to load resource-specific content.

Common use cases include:

- user profile pages
- product details
- order details
- blog posts

Structural Example

Rules

- Dynamic route parameters must always be validated.
- The application must never assume parameter validity.
- Invalid parameters must result in a proper error or "not found" response.
- Resource lookups must always be validated server-side.

7.5 Query-Supported Routes

Query parameters allow passing additional state through the URL.

They are commonly used for:

- pagination
- filtering
- sorting
- searching
- UI state persistence

Structural Example

```
src
 └── app
     ├── users
     │   └── page.tsx
     │
     ├── products
     │   └── page.tsx
     │
     └── orders
         └── page.tsx
```

Example URLs that may use query parameters:

```
/users?page=1
/products?category=laptop
/orders?status=completed
```

Rules

- Query parameters must always be validated.
- Default values must be defined when parameters are missing.
- UI state should remain synchronized with URL query parameters.
- Query parameters must never contain sensitive data.

8. State Management

Use Case	Solution
Component-only state	`useState`
Shared UI state	Context
App-wide state	Redux / Zustand
Server state	React Query / TanStack Query

- **Store structure**
    - **Zustand (Recommended)**

```
Zustand – Global Store Structure

store/
├── index.ts              → Exports all global Zustand stores
├── auth.store.ts         → Manages global authentication state
├── theme.store.ts        → Manages global theme state (light/dark/system)
```

index.ts (Aggregator Pattern)

```typescript
export * from "./auth.store";
export * from "./theme.store";
```

Rules:

- The `store` folder must contain **only global-level Zustand stores**
- Global stores represent cross-application concerns (e.g., auth, theme, app config)
- Feature-specific or page-level state must **not** be added here
- Each store must be **independent and modular**

- UI components must consume state via the exported Zustand hooks only

- **Redux Toolkit (RTK)**

```
RTK (Redux Toolkit) – Global Store Structure

store/
├── index.ts                → Main Redux store configuration
├── slices/
│   ├── index.ts            → Combines all global slices and creates the root reducer
│   ├── auth.slice.ts       → Manages global authentication state
│   ├── theme.slice.ts      → Manages global theme state (light/dark/system, etc.)
```

Rules:

- The `slices` folder must contain only global-level slices
- Global slices represent **cross-application concerns** (e.g., auth, theme, app config)
- Feature-specific or page-level state must **not** be added here
- All slices must be imported and combined **only** in `slices/index.ts`

---

9. API & Networking

API Architecture Overview

The application must follow a **two-layer API architecture** inside the `/services` directory:

```
Plain text

/services
  client.ts   // Axios client, interceptors, query client
  api.ts      // All API endpoints using the wrapped client
```

client.ts (API Client Layer)

- `client.ts` is the **single, centralized API client**
- It is responsible for:
    - Axios instance creation
    - Base URL configuration
    - Request and response interceptors
    - Authentication headers
    - Global error handling

Rules:

- Axios (or Fetch) must be used **only inside `client.ts`**
- No other file is allowed to create or configure HTTP clients
- No business logic is allowed in this file

---

api.ts (Endpoint Definition Layer)

- `api.ts` must contain **all API endpoint definitions**

- All endpoints must use the wrapped client from `client.ts`
- Endpoints must be:
    - Clearly named
    - Grouped logically (auth, user, product, etc.)
    - Typed (request and response)

Rules:

- No direct Axios or Fetch usage inside `api.ts`
- No API logic inside UI or container components
- No additional service files are allowed

---

10. Error Handling

Error handling must be **centralized, predictable, and user-safe**.
The application must never crash silently or expose internal error details to users.

10.1 Error Boundaries

Error Boundaries are used to catch **runtime UI errors** and prevent the entire app from crashing.

Rules

- A **global Error Boundary is mandatory** at the application root
- Feature-level Error Boundaries are allowed for critical modules
- Error Boundaries **must not** contain business logic
- Error Boundaries **must not** retry API calls automatically

Responsibilities

- Catch rendering errors
- Show fallback UI
- Log the error for monitoring

Example Responsibilities

- Display a generic “Something went wrong” screen
- Provide a refresh or navigation option
- Capture error stack trace for logging

10.2 API Errors

API errors must be handled **outside UI components** and normalized before reaching the UI.

Rules

- All API calls must go through a **central API client**
- API error handling must be implemented using:
    - Interceptors (Axios) or
    - Central fetch wrapper
- UI components must **never parse raw API error responses**

Standard API Error Categories

- **400** – Validation / bad request
- **401** – Unauthorized → logout or token refresh
- **403** – Forbidden → access denied screen
- **404** – Resource not found
- **500+** – Server error

Mandatory Practices

- Normalize API error shape before returning
- Map technical errors to user-friendly messages
- Handle auth-related errors globally

10.3 Toast Handling

Toasts are used only for **user-visible, short-lived feedback**.

Allowed Use Cases

- API success messages
- API failure messages
- Form submission feedback
- Non-blocking warnings

Rules

- Toast logic must be centralized (utility or hook)
- UI components must not hardcode toast messages
- Avoid duplicate toasts for the same event
- No toast spamming (rate-limit repeated failures)

Guidelines

- Success → short and clear
- Error → user-friendly, no technical jargon
- Critical errors → modal or error screen (not toast)

10.4 Logging

Logging is for **developers and monitoring systems**, not users.

Rules

- `console.log`, `console.error` are **forbidden in production**
- Use a centralized logging utility
- Log only actionable and meaningful data
- Never log:
    - Tokens
    - Passwords
    - Personal or sensitive user data

What to Log

- Application crashes
- Unhandled exceptions
- API failures (with metadata)
- Performance issues (optional)

Logging Levels

- `info` – lifecycle events
- `warn` – recoverable issues
- `error` – crashes and failures

---

11. Custom Hooks Guidelines

Custom hooks are used to **extract reusable logic** and **share behavior across components**. They must remain **pure, predictable, and UI-agnostic**.

11.1 Naming

Rules

- Hook names **must start with** `use`
- Use **camelCase** after `use`
- Name must clearly describe behavior or intent

Examples

- `useAuth`
- `useDebounce`
- `useUserPermissions`

File Naming

- File name must match kebab-case (use-auth.ts, use-debounce.ts, etc.)
- One hook per file

11.2 Reusability

Hooks must be **generic and reusable** by default.

Rules

- Hooks must not depend on a specific page or feature
- Avoid hardcoded values
- Accept configuration via parameters
- Hooks should return:
    - State
    - Derived data

    - Action handlers

Good Practice

- Prefer small, composable hooks
- Split large hooks into multiple focused hooks
- Hooks should work in **any component context**

11.3 No UI Logic

Custom hooks must contain **zero UI logic**.

Strict Rules

- No JSX
- No HTML elements
- No styling logic
- No direct DOM manipulation

Allowed Inside Hooks

- State management
- Side effects (`useEffect`)
- API calls (via services)
- Data transformation
- Event handlers

Not Allowed Inside Hooks

- Rendering components

- Opening modals
- Showing toasts directly
- Navigating routes directly

Hooks provide data and actions, and UI decides how to present them.

11.4 Testing Hooks

Custom hooks must be **testable and predictable**.

Rules

- Hooks with business logic **must have tests**
- Hooks must not rely on hidden globals
- External dependencies must be mockable

Testing Guidelines

- Test behavior, not implementation
- Cover:
    - Initial state
    - State updates
    - Side effects
    - Error handling

Recommended Tools

- Jest or Vitest (Recommended)
- React Testing Library (`renderHook`)

---

12. Business Logic & UI Separation

This architecture separates **what the app does**, **how screens are coordinated**, and **how UI is rendered**.
The goal is to keep logic testable, UI reusable, and screens easy to reason about.

12.1 Business Logic

Business Logic defines **data handling, rules, and state management**.
It includes fetching data, transforming responses, validations, and managing shared state.
In this project, business logic lives **inside container hooks and store files**, but **never in JSX**.

```
containers/user-management/
├─ users/
│  └─ use-users.ts
├─ add-edit-user/
│  └─ use-add-edit-user.ts
└─ use-user-store.ts
```

12.2 Container Layer (Screen coordination)

The Container Layer coordinates screens.
It connects business logic to UI components, handles lifecycle events, and passes prepared data to UI.
Containers compose screens but do not contain business rules or UI-specific rendering logic.

```
containers/user-management/
├─ users/
│  └─ index.tsx
├─ add-edit-user/
│  └─ index.tsx
```

12.3 UI Layer (Pure UI)

The UI Layer contains **pure presentational components**.
Components only render data received via props and emit user interactions via callbacks.
They do not access APIs, stores, or business logic directly.

```
components/user-management/
├─ user-actions.tsx
└─ user-card.rtsx
```

Golden Rule:

- All data fetching must live inside container hooks (`use-users.ts`, `use-add-edit-user.ts`)
- UI components must not access stores directly
- UI components must not import container hooks
- Pages must only render containers, nothing else
- State management must be handled via hooks or stores
- UI components receive data and callbacks via props only
- Each screen must have a single container entry point
- Hooks must return data and handlers, never JSX

---

13. UI & Theming

13.1 Styling System

- The project must use a **single, standardized styling system**.
- Allowed styling approaches:

    - Tailwind CSS (Latest version recommended)
    - CSS Modules
    - Styled Components (only if approved)
- Mixing multiple styling systems in the same project is **not allowed**.
- Inline styles should be avoided except for dynamic values.

13.2 Theme Provider

- A **central Theme Provider** is mandatory.
- The theme provider must:
    - Control colors, typography, spacing, and shadows
    - Be initialized at the application root
- UI components must consume theme values instead of hardcoded styles.

13.3 Design Tokens

- Design tokens must be defined centrally.
- Tokens include:
    - Colors
    - Font sizes
    - Spacing
    - Border radius
    - Z-index levels
- Tokens must be reusable and consistent across the application.
- Hardcoded UI values inside components are **not allowed**.

13.4 Dark Mode

- Dark mode support is recommended for all new projects.
- Theme switching must be handled via the Theme Provider.
- Components must not contain hardcoded light or dark colors.
- User preferences must be respected and persisted.

---

14. Common Components

Common components are **pure UI building blocks** used across the application.

- Buttons
- Inputs
- Tables
- Modals

Example:

```
components/ui
├─ input.tsx
├─ table.tsx
├─ avatar.tsx
└─ button.tsx
```

Rules & Responsibility

- Components must be **fully reusable**
- No business logic inside common components
- No API calls inside components
- Common components handle **presentation only**

- Data fetching and state management belong to containers
- Validation and side effects must be handled outside UI components
- Props must be clearly typed
- Styling must rely on the theme and design tokens
- Components must remain framework-agnostic where possible

---

15. Performance Optimization

Performance optimization must be considered by default for all **Next.js applications**, especially for large-scale and user-facing systems.

Next.js provides built-in performance features such as:

- Server Components
- Automatic code splitting
- Streaming
- Image optimization (next/image)
- Font optimization (next/font)
- Incremental Static Regeneration (ISR)

Developers must design both **server-side and client-side rendering paths** carefully to avoid unnecessary computation, network overhead, and large client bundles.

15.1 Server Components First Strategy

Next.js uses **Server Components by default** in the App Router.
Server Components reduce the amount of JavaScript sent to the browser and improve performance.

Server Components should handle:

- data fetching
- data transformation
- heavy computations
- server-side rendering logic

Client Components should only be used when browser interactivity is required.

Rules

- Server Components must be preferred whenever possible.
- Client Components must be used only for interactive UI.
- Heavy logic should be executed on the server.
- Minimize client-side JavaScript sent to the browser.

15.2 Automatic Code Splitting

Next.js automatically splits application code based on routes and route segments.

Each page loads only the code required for that route, reducing the initial bundle size.

Feature modules should also be structured to prevent unnecessary bundling of unrelated functionality.

Rules

- Route-level code splitting must rely on Next.js automatic behavior.

- Large feature modules should be separated logically.
- Unrelated modules must not be bundled together.
- Client-side bundles should remain minimal.

15.3 Dynamic Component Loading

Some components should be loaded **only when required** to reduce the initial page load cost.

Typical examples include:

- analytics tools
- charts and data visualization
- complex dashboards
- optional UI modules

Rules

- Heavy client components should be loaded dynamically.
- Non-critical components must not block the initial page render.
- Dynamic loading should reduce the initial JavaScript payload.

15.4 Image Optimization

Next.js provides a built-in image optimization system that improves loading performance and responsiveness.

Images should be automatically optimized for:

- responsive sizes

- modern formats
- lazy loading
- bandwidth efficiency

Rules

- Images must use the Next.js image optimization system.
- Static images should be stored inside the project’s public assets directory.
- Responsive images must be used for large screens and mobile devices.
- Unoptimized images should not be used in production environments.

15.5 Font Optimization

Next.js includes a built-in font optimization system that improves loading performance and prevents layout shifts.

Fonts should be loaded efficiently to avoid blocking page rendering.

Rules

- Fonts must be loaded using the Next.js font optimization system.
- Avoid loading multiple unnecessary font families.
- Font files must be optimized for performance.
- Font loading must avoid layout shifts.

15.6 Caching & Revalidation Strategy

Next.js supports advanced caching mechanisms that allow balancing performance and data freshness.

Common strategies include:

- static rendering
- server rendering
- incremental static regeneration
- revalidation

Applications must choose the appropriate strategy depending on the data requirements.

Rules

- Static rendering should be preferred for stable content.
- Incremental static regeneration should be used for semi-dynamic content.
- Server rendering should be used for user-specific or frequently changing data.
- Caching strategies must be documented for critical routes.

15.7 Monitoring & Measurement

Performance optimization must be based on measurement and profiling rather than assumptions.

Applications must be monitored regularly to detect performance regressions.

Recommended tools include:

- Browser performance tools
- Lighthouse audits

- Next.js build analysis tools
- server monitoring tools
- Vercel analytics and monitoring tools

Rules

- Performance optimizations must be data-driven.
- Bundle sizes must be monitored regularly.
- Performance regressions must be treated as high-priority issues.
- Critical user flows should be periodically audited.

---

16. Security Guidelines

Security rules apply across **Server Components, Client Components, API routes, middleware, and services**.

Security violations must be treated as **blockers during code review**.

Applications must ensure that sensitive data, authentication tokens, and internal APIs are never exposed unnecessarily to the client.

16.1 Token Storage

Authentication tokens must be handled securely to prevent unauthorized access and session hijacking.

Next.js applications should manage authentication tokens primarily through **secure server-managed cookies**.

Rules

- Authentication tokens must not be stored inside UI components.

- Tokens should be stored in **HttpOnly cookies** whenever possible.
- Sensitive tokens must not be accessible through client-side JavaScript.
- Avoid storing authentication tokens in `localStorage` or `sessionStorage`.
- Refresh tokens must never be stored on the client.
- Authentication tokens must never be logged in application logs.
- Tokens must never appear in error messages or debugging output.
- Authentication validation should be handled through **middleware, server components, or API routes**.

16.2 XSS Prevention

Cross-Site Scripting (XSS) occurs when malicious scripts are injected into web pages through untrusted content.

Next.js applications must treat **all user input and external data as untrusted**.

Rules

- Avoid rendering raw HTML content unless necessary.
- Dynamic HTML content must be sanitized before rendering.
- User-generated input must always be validated.
- API responses must not be trusted blindly.
- Input validation must occur on both client and server.
- Sensitive content must not be rendered directly from untrusted sources.
- Applications must rely on framework-level escaping for UI rendering.

16.3 API Protection

Next.js applications expose backend logic through **API routes or server-side handlers**.

These endpoints must be protected to prevent unauthorized access or data leaks.

Rules

- All API routes must validate authentication and authorization.
- Request payloads and parameters must be validated before processing.
- Sensitive business logic must run on the server side only.
- Internal system details must never be exposed in API responses.
- Client components must not contain sensitive backend logic.
- Authorization checks must occur before protected operations.
- Error responses must not expose stack traces or internal server details.

16.4 Environment Variables

Environment variables provide configuration for different environments such as development, staging, and production.

Improper handling of environment variables can expose sensitive credentials.

Rules

- Real `.env` files must never be committed to version control.
- A `.env.example` file must be maintained for required variables.
- Secrets must remain accessible only on the server side.
- Only environment variables intended for the client should be publicly exposed.
- Sensitive credentials must not be embedded in frontend code.

- Environment variables must not be hardcoded in the application.
- Each environment must maintain separate configuration values.

---

17. Testing Standards

Testing is mandatory for **business logic and containers**. UI components should be tested where behavior matters.

17.1 Jest or Vitest (Recommended)

- Test business logic, hooks, stores, and containers directly
- No snapshot-only tests
- Focus on user behavior and interactions
- Mock external dependencies and APIs
- Do not test implementation details

17.2 React Testing Library

- Use React Testing Library for **component and container tests**
- Test **user behavior**, not component internals
- Prefer queries like:
  - `getByRole`
  - `getByText`
  - `getByLabelText`
- Avoid testing internal state or private methods